Advice

Student Name: David Hadzhiivanov

Student Number: 5670330

Table of Contents

Advice 1
Table of Contents 1
1. Technical Documentation 1
1.1 Project Overview 1
1.2 Core Components 1
1.3 Key Features 1
2. Product Comparisons 1
3. Advisory Reporting 2
3.1 Risks & Ethical Considerations 2
3.2 Mitigation Strategies 2
4. Improvement Proposals 2
4.1 Software Enhancements 3
5. Advice Summary 3
5.1 Recommended Development Path 3
5.2 Suggested Tools & Libraries 3
6. Future considerations 3
7. Conclusion 4

1. Technical Documentation

1.1 Project Overview

A secure notepad-style web application that lets users write, save, edit, and retrieve notes. Notes are encrypted before storage; The backend is Flask (Python), frontend is HTML/CSS, data stored in SQLite, hosted on a Raspberry Pi behind NGINX (with eventual NetLab consideration.

1.2 Core Components

- **Frontend:** HTML/CSS.
- **Backend:** Flask application serving API endpoints for auth and note operations.
- **Database:** SQLite for user data and encrypted note blobs.
- **Encryption layer:** Symmetric encryption (AES) for note contents; asymmetric/key-management considerations for key exchange if needed.
- **Hosting:** Raspberry Pi with NGINX as reverse proxy; HTTPS via Let's Encrypt/manual certs.
- **Utilities:** Werkzeug Security for password hashing; cryptography / PyCryptodome for AES routines.

1.3 Key Features

- Account creation and secure login (hashed passwords).
- Client- or server-assisted encryption workflow so server cannot decrypt notes without user consent.
- HTTPS enforced for all traffic.
- Lightweight, self-hostable on Raspberry Pi for practical server management experience.

2. Product Comparisons

Quick comparison of the alternatives considered or advised on:

- **Hosting: Raspberry Pi vs NetLab**
- *Raspberry Pi:* full control, ideal for learning NGINX/SSL setup, portable; risk = hardware reliability and public availability (requires port forwarding), supports all types of web applications.
- *NetLab:* managed infrastructure, stable public hosting, may have restrictions on custom SSL or port configs, supports only PHP, Wordpress and pure HTML.
- **Encryption: AES vs RSA**
- *AES (symmetric):* fast, handles large data (recommended for notes).
- *RSA (asymmetric):* slow, not suitable for large payloads - useful for encrypting keys or signatures.
- **Datastore: SQLite vs external DB**
- *SQLite:* simple, file-based, low overhead (good for prototype).

- *External DB (Postgres/MySQL):* better for scaling, more robust concurrency - overkill for the current scope.
- **Hashing: SHA-256 vs Werkzeug**
- *Raw SHA-256:* basic hashing - not sufficient alone.
- *Werkzeug Security:* salted, tested helper functions.

Using the Raspberry Pi + AES + SQLite + Werkzeug for this project phase balances learning goals and feasibility.

3. Advisory Reporting

3.1 Risks & Ethical Considerations

- **Data exposure risk:** If keys are mishandled, encrypted notes may become irrecoverable or decryptable by attackers.
- **Hosting exposure:** Opening ports and self-hosting exposes a device to internet threats.
- **Misuse:** While a notepad is benign, any insecure storage system that leaks sensitive data has real consequences for users.
- **Legal/Privacy:** Storing personal or sensitive data requires responsibility.

3.2 Mitigation Strategies

- Keep keys off the public server; store them encrypted on the Pi or require client-side passphrase-derived keys.
- Harden the Raspberry Pi: firewall, fail2ban, minimal services, regular updates.
- Use verified SSL certificates and HTTPS-only traffic.
- Limit attack surface: disable unused ports, avoid running services as root.

4. Improvement Proposals

4.1 Software Enhancements

- **Key management:** Use passphrase-derived keys (PBKDF2 / scrypt) on the client or store encrypted keys on the Pi with an HSM-like approach for later.
- **Better error handling & logging:** Structured logs for auth attempts and encryption operations.

5. Advice Summary

5.1 Recommended Development Path

- Finalize **HTTPS setup** on the Raspberry Pi (manual or Let's Encrypt).
- Complete **authentication system** using Werkzeug Security.
- Implement **AES-based encryption** or simpler **substitution encryption** for notes and ensure proper key handling.
- Conduct **functional and security testing** (authentication, encryption, HTTPS).
- Present the working demo with emphasis on encryption, authentication, and secure deployment.

5.2 Suggested Tools & Libraries

Tool / Library	Purpose	Reason
Flask	Web framework	Lightweight, easy for small secure apps
SQLite	Database	Simple file-based storage
Werkzeug security	Password hashing	Secure and easy to integrate
Cryptography / PyCryptodome	Encryption	AES implementation support
NGINX	Reverse proxy	Handles SSL/TLS and port management
Certbot	HTTPS certificates	Handles obtaining the certificates for secure HTTPS connection between client and server
Raspberry Pi	Hosting	Offers real-world deployment practice

6. Future considerations

- Implement **client-side encryption** for full end-to-end protection.
- Implement **2 Factor Authentication (2FA)** for account security.
- Add **key recovery mechanism** if users lose credentials.
- Introduce **user roles** for better access control.

7. Conclusion

The project’s advisory process established a clear technical foundation for building a secure and practical web-based notepad. Through careful comparison of tools, encryption standards, and hosting options, the chosen stack — Flask, SQLite, AES (or substitution encryption for simplicity), and a Raspberry Pi deployment with HTTPS — provides the right balance between feasibility and educational value.

This configuration demonstrates a hands-on understanding of security principles: data confidentiality through encryption, authentication integrity via hashed credentials, and secure communication using SSL/TLS. The advice also highlights the importance of controlled hosting and responsible handling of encryption keys, aligning the implementation with ethical cybersecurity standards.